

Data Structures: Arrays

Data Structure Course
DGIST

November 1, 2025

Contents

1. Introduction to Arrays
2. Static vs Dynamic Arrays
3. Amortized Analysis
4. Array Operations
5. Multidimensional Arrays
6. Memory Layout & Cache Locality
7. Applications & Limitations
8. Summary

Introduction to Arrays

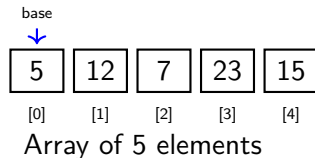
What is an Array?

Definition

An **array** is a contiguous memory block that stores elements of the same type, supporting **$O(1)$ indexing**.

Key characteristics:

- Fixed or dynamic size
- Elements stored consecutively in memory
- Direct access via index
- Cache-friendly data structure
- Foundation for many other data structures



Memory Address Calculation

Address of `arr[i]` = `base_address` + `i` × `sizeof(element)`

Static vs Dynamic Arrays

Static Arrays

Definition

Fixed size determined at compile time or initialization

Advantages:

- No resize overhead
- Predictable memory usage
- Slightly faster access
- Simple memory management

Disadvantages:

- Cannot grow or shrink
- Must know size in advance
- Wasted space if overallocated

Examples

C:

```
1 int arr[100]; // Fixed size
```

Java:

```
1 int[] arr = new int[100];
```

C++:

```
1 int arr[100];  
2 std::array<int, 100> arr;
```

Use Case

Dynamic Arrays

Definition

Resizable arrays that grow as needed

Growth Strategy:

- Start with small capacity
- Double capacity when full (typical)
- Allocate new array and copy elements
- Free old array

Examples:

- C++: `std::vector`
- Java: `ArrayList`
- Python: `list`
- C#: `List<T>`

Python Implementation

```
1 class DynamicArray:
2     def __init__(self):
3         self.capacity = 1
4         self.size = 0
5         self.array = [None] * self.
            capacity
6
7     def append(self, item):
8         if self.size == self.capacity:
9             self._resize()
10        self.array[self.size] = item
11        self.size += 1
12
13    def _resize(self):
14        self.capacity *= 2
15        new_array = [None] * self.capacity
16        for i in range(self.size):
17            new_array[i] = self.array[i]
18        self.array = new_array
```

Comparison: Static vs Dynamic Arrays

Operation	Static Array	Dynamic Array
Access	$O(1)$	$O(1)$
Append	N/A	$O(1)$ amortized
Insert (middle)	$O(n)$	$O(n)$
Delete (middle)	$O(n)$	$O(n)$
Memory	Exact	1.5-2× actual size
Resize	No	Yes (expensive)

Static Arrays Best For

- Fixed-size data
- Memory-constrained systems
- Real-time systems

Dynamic Arrays Best For

- Unknown or changing size
- Frequent additions
- General-purpose use

Amortized Analysis

Dynamic Array Resizing Cost

Resizing Strategy

When array is full, allocate new array with $2\times$ capacity

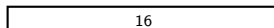
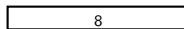
Single Resize Cost: $O(n)$

- Allocate new array: $O(\text{capacity})$
- Copy all n elements: $O(n)$
- Free old array: $O(1)$

But resizes are infrequent!

- Resizes at sizes: 1, 2, 4, 8, 16, ..., n
- Total resizes: $\log_2 n$
- Total copy cost: $1 + 2 + 4 + \dots + n = 2n - 1$

Growth Sequence



...

Amortized Analysis Result

Accounting Method for Amortized Analysis

Intuition

Charge extra "credits" for cheap operations to pay for expensive ones

Charge \$3 per insertion:

- \$1 for actual insertion
- \$2 saved as credit for future resize

When resize happens:

- Need to copy n elements
- Each element has \$2 credit saved
- Total credits: $2n$
- Resize cost: $\$n$ (copying)
- Credits cover the cost!

Growth Factor Comparison

Factor	Memory	Frequency
$1.5\times$	Lower	More
$2\times$	Higher	Less

Shrinking Strategy

Shrink when $\text{size} < \text{capacity}/4$
(not $\text{capacity}/2$ to avoid thrashing)

Trade-off: Space efficiency vs resize frequency

Array Operations

Access and Search Operations

Access by Index: $O(1)$

```
1 element = arr[i]
```

Direct memory calculation:
 $\text{base} + i \times \text{sizeof}(\text{element})$

Linear Search: $O(n)$

```
1 def linear_search(arr, target):  
2     for i in range(len(arr)):  
3         if arr[i] == target:  
4             return i  
5     return -1
```

Check each element sequentially

Binary Search: $O(\log n)$

Only for sorted arrays!

```
1 def binary_search(arr, target):  
2     left, right = 0, len(arr) - 1  
3     while left <= right:  
4         mid = left + (right - left) // 2  
5         if arr[mid] == target:  
6             return mid  
7         elif arr[mid] < target:  
8             left = mid + 1  
9         else:  
10            right = mid - 1  
11    return -1
```

Key Insight

Binary search eliminates half the search space each iteration

Insert and Delete Operations

Insert at Position i: $O(n)$

```
1 def insert(arr, index, value):
2     arr.append(None) # Ensure space
3     # Shift elements right
4     for i in range(len(arr)-1, index, -1):
5         arr[i] = arr[i-1]
6     arr[index] = value
```

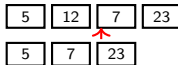
Insert 99 at index 2



Delete at Position i: $O(n)$

```
1 def delete(arr, index):
2     # Shift elements left
3     for i in range(index, len(arr)-1):
4         arr[i] = arr[i+1]
5     arr.pop() # Remove last
```

Delete at index 1



Performance Summary

- Insert/delete at end: $O(1)$ amortized
- Insert/delete at beginning: $O(n)$ - worst case
- Insert/delete in middle: $O(n)$ - must shift elements

Multidimensional Arrays

2D Arrays (Matrices)

Declaration

Python:

```
1 matrix = [[0]*cols for _ in range(rows)]
```

Java:

```
1 int [][] matrix = new int[rows][cols];
```

C++:

```
1 vector<vector<int>> matrix(rows,  
2     vector<int>(cols));
```

3x4 Matrix

0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

matrix[i][j]

Accessing Elements

element = matrix[i][j]

Time: O(1)

Memory Layout: Row-Major vs Column-Major

Row-Major Order

Used in: C, C++, Python, Java

Store rows contiguously:

[row0] [row1] [row2] ...

Address calculation:

$\text{base} + (i \times \text{cols} + j) \times \text{size}$

Better for:

- Iterating by rows
- Row-wise operations

Column-Major Order

Used in: Fortran, MATLAB

Store columns contiguously:

[col0] [col1] [col2] ...

Address calculation:

$\text{base} + (j \times \text{rows} + i) \times \text{size}$

Better for:

- Iterating by columns
- Column-wise operations

Performance Impact

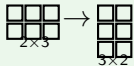
Accessing memory in the wrong order can cause 10-100× slowdown due to cache misses!

Common Matrix Operations

Transpose: $O(\text{rows} \times \text{cols})$

```
1 def transpose(matrix):
2     rows = len(matrix)
3     cols = len(matrix[0])
4     result = [[0]*rows for _ in range(cols)]
5     for i in range(rows):
6         for j in range(cols):
7             result[j][i] = matrix[i][j]
8     return result
```

Example



Rotate 90° Clockwise

```
1 def rotate_90(matrix):
2     n = len(matrix)
3     # 1. Transpose
4     for i in range(n):
5         for j in range(i, n):
6             temp = matrix[i][j]
7             matrix[i][j] = matrix[j][i]
8             matrix[j][i] = temp
9
10    # 2. Reverse each row
11    for i in range(n):
12        matrix[i].reverse()
```

Matrix Multiplication

Naive: $O(n^3)$

Strassen: $O(n^{2.37})$

Coppersmith-Winograd: $O(n^{2.376})$

Memory Layout & Cache Locality

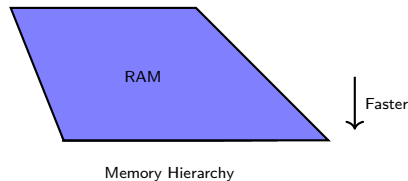
Cache Hierarchy and Performance

Memory Hierarchy

Level	Size	Latency
L1 Cache	32-64 KB	1-4 cycles
L2 Cache	256 KB-1 MB	10-20 cycles
L3 Cache	8-32 MB	40-75 cycles
RAM	GB	200+ cycles

Cache Line

- Typical size: 64 bytes
- Fetching one element loads entire cache line
- Sequential access benefits from prefetching



Spatial Locality: Good vs Bad Access Patterns

Good: Sequential Access

```
1 # Excellent cache locality
2 sum = 0
3 for i in range(n):
4     sum += arr[i]
```

- Accesses consecutive memory
- Cache prefetching works well
- Minimizes cache misses

Bad: Random Access

```
1 # Poor cache locality
2 sum = 0
3 for i in random_indices:
4     sum += arr[i]
```

- Jumps around memory
- No benefit from prefetching
- Frequent cache misses

Matrix Traversal Example (C/C++ row-major)

```
1 // GOOD: Row-wise iteration (follows memory layout)
2 for (int i = 0; i < rows; i++)
3     for (int j = 0; j < cols; j++)
4         sum += matrix[i][j];
```

Optimization: Array of Structures vs Structure of Arrays

Array of Structures (AoS)

```
1 struct Point {  
2     float x, y, z;  
3 };  
4 Point points[1000];  
5  
6 // Access x coordinates  
7 for (int i = 0; i < 1000; i++)  
8     sum += points[i].x;
```

Memory layout:

x0 y0 z0 | x1 y1 z1 | x2 y2 z2 ...

Issue: Loading unnecessary y, z values

Structure of Arrays (SoA)

```
1 struct Points {  
2     float x[1000];  
3     float y[1000];  
4     float z[1000];  
5 };  
6 Points points;  
7  
8 // Access x coordinates  
9 for (int i = 0; i < 1000; i++)  
10     sum += points.x[i];
```

Memory layout:

x0 x1 x2 ... | y0 y1 y2 ... | z0
z1 z2 ...

Benefit: Better cache locality for single field

Applications & Limitations

When to Use Arrays

Common Applications

- **Lookup tables:** ASCII mappings, precomputed values
- **Buffers:** Fixed-size data storage, I/O buffers
- **Stacks/Queues:** Array-based implementations
- **Dynamic programming:** Memoization tables
- **Sorting algorithms:** In-place sorting
- **Image processing:** Pixel arrays (2D/3D)
- **Scientific computing:** Vectors, matrices, tensors

Advantages

- ✓ $O(1)$ random access by index
- ✓ Cache-friendly (contiguous memory)
- ✓ Simple and intuitive
- ✓ Low memory overhead
- ✓ Excellent for iteration

Limitations

- ✗ Fixed size or resize overhead
- ✗ $O(n)$ insertion/deletion in middle
- ✗ Wasted space if sparse
- ✗ Cannot efficiently grow in middle

When to Use Alternatives

Use Case	Better Alternative	Reason
Frequent insertions/deletions	Linked List	$O(1)$ insert/delete
Unknown size, many insertions	Dynamic Array	Amortized $O(1)$ append
Key-value lookups	Hash Table	$O(1)$ average lookup
Sorted data with insertions	Binary Search Tree	$O(\log n)$ operations
Priority queue	Heap	$O(\log n)$ insert/extract-min
Sparse data	Hash Table / Sparse Matrix	Space efficient

Summary

Key Takeaways

Array Fundamentals

- Contiguous memory blocks with $O(1)$ indexing
- Foundation for many data structures
- Direct memory address calculation

Static vs Dynamic

- Static: fixed size, no overhead, predictable
- Dynamic: resizable, $O(1)$ amortized append, flexible
- Amortized analysis shows $O(1)$ average cost for dynamic growth

Performance Considerations

- Cache locality is critical: $10\text{-}100\times$ performance difference
- Access patterns matter: sequential $>$ random
- Row-major vs column-major order affects cache performance

Thank You!

Questions?